

A Novel Digital Watermarking Framework For Any Binary File (Image/Audio/Text) Using Hash-Derived Spatial Distribution

Project Submitted By

Aparajito Ray Chaudhuri

Akash Chakraborty

Abhirup Samadder

Sanjukta Sarkar

BACHELOR OF TECHNOLOGY (CSE)



DEPARTMENT OF COMPUTER SCIENCE

Sister Nivedita University, Newtown

Kolkata, West Bengal

2026

A Novel Digital Watermarking Framework For Any Binary File (Image/Audio/Text) Using Hash-Derived Spatial Distribution

Project Submitted in Partial Fulfilment of the Requirements for
Award of the Degree of

Bachelor of Technology (CSE)

Student Name	Reg. Number	Email ID
Aparajito Ray Chaudhuri	220100142234	apara1807@gmail.com
Akash Chakraborty	220100087188	akashchakraborty694@gmail.com
Abhirup Samadder	220100624372	samadderabhirup2004@gmail.com
Sanjukta Sarkar	220100439323	sanjuktasarkar64@gmail.com

Submission Date: May 08, 2026

Under the supervision of

Prof. Atal Chaudhuri

Emeritus Professor

Department of Computer Science

Sister Nivedita University, Newtown

Kolkata, West Bengal



DEPARTMENT OF COMPUTER SCIENCE

Sister Nivedita University, Newtown

Kolkata, West Bengal

May, 2026

Declaration

We hereby declare that this dissertation is our original work. Any idea, data, or reference taken from another source has been properly cited in the report. This project has not been submitted earlier, either fully or partly, to any other university or institution for any degree, diploma, or academic award.

Date: 08-05-2026

Place: SNU, WB

.....
Aparajito Ray Chaudhuri

Registration Number: 220100142234

.....
Akash Chakraborty

Registration Number: 220100087188

.....
Abhirup Samadder

Registration Number: 220100624372

.....
Sanjukta Sarkar

Registration Number: 220100439323



Certificate

This is to certify that the project entitled "A Novel Digital Watermarking Framework for Any Binary File (Image/Audio/Text) Using Hash-Derived Spatial Distribution", submitted by Aparajito Ray Chaudhuri, Akash Chakraborty, Abhirup Samadder, and Sanjukta Sarkar to Sister Nivedita University, West Bengal, for the award of the degree of Bachelor of Technology (CSE) is a bonafide record of the project work carried out by them under my supervision and guidance. To the best of my knowledge, this work has not been submitted earlier, either fully or partly, to any other institute or university for the award of any degree or diploma.

Date: 08-05-2026

Place: SNU, WB

.....
(Prof. Atal Chaudhuri)
Emeritus Professor
Dept. of Computer Science
Sister Nivedita University

Contents

List of Tables	vii
List of Figures	viii
Acknowledgement	ix
Abstract	x
1 Introduction	1
1.1 Context of the Study	1
1.2 Objectives	2
1.3 Scope and Organization	3
1.4 Contributions	3
2 Literature Survey	5
2.1 Introduction to Literature Survey	5
2.2 Foundations of Information Hiding	5
2.3 Existing Systems and Related Work	6
2.3.1 Image Watermarking and Spatial-Domain LSB Methods	6
2.3.2 Image Watermarking in the Transform Domain	6
2.3.3 Audio Steganography	6
2.3.4 Text Steganography	7
2.3.5 Video Watermarking and Steganography	7
2.3.6 Cryptographic Primitives	7
2.4 Multi-Modal and Multi-Watermark Architectures	8
2.4.1 Rationale for Multi-Modal Design	8
2.4.2 Single-Watermark LSB Systems	8
2.4.3 Multi-Watermark Schemes	8
2.5 Comparative Evidence from Literature	9
2.6 Limitations of Existing Systems	9
2.6.1 General Modeling Limitations	9
2.6.2 Multi-Watermark and Integration Gaps	10
2.7 Comparative Study and Gap Analysis	10
2.8 Position of the Proposed Work	10
2.9 Summary of Literature Review	11
3 Problem Identification	12
3.1 Problem Statement	12
3.2 Formal Statement of the Problem	12
3.3 Research Gap to be Solved	13
3.4 Why Existing Methods Are Insufficient	13
3.5 Research Objectives and Hypotheses	13

3.6	Threat Model and Assumptions	14
3.7	Design Constraints and Requirements	15
3.8	Expected Challenges	16
3.9	Identified Sub-Problems	16
4	Proposed Methodology	18
4.1	System Architecture Overview	18
4.1.1	Typical User Flow	20
4.1.2	Why the System is Organized this Way	20
4.2	Image-Watermarking Module	20
4.2.1	Password-to-Location Mapping	21
4.2.2	Per-Pixel 3–3–2 Bit Split	21
4.2.3	Row-Occupation Marker	21
4.2.4	Self-Delimited Payload Layout	22
4.2.5	Embedding Algorithm	22
4.2.6	Extraction Algorithm	23
4.2.7	Worked Example	24
4.3	Audio-Steganography Module	25
4.4	Text-Steganography Module	25
4.5	Video-Steganography Module	26
4.6	Common Payload Format and Timestamping	27
4.7	Computational Complexity	27
4.8	Evaluation Metrics	28
4.8.1	Perceptual-Quality Metrics	28
4.8.2	Capacity Metrics	28
4.8.3	Correctness Metrics	28
4.8.4	Run-Time Metrics	29
4.8.5	Comparison Protocol	29
4.9	Implementation Details	29
4.9.1	Development Environment	29
4.9.2	Computational Resources	29
4.9.3	Reproducibility	29
4.10	Summary	30
5	Results and Discussion	32
5.1	Experimental Setup	32
5.1.1	Hardware and Software Environment	32
5.1.2	Cover Files Used	33
5.1.3	Evaluation Procedure	33
5.2	Module Performance Evaluation	34
5.2.1	Image Module: Embedding and Extraction	34
5.2.2	Image Module: Multi-Watermark Capacity	35
5.2.3	Audio Module	35
5.2.4	Text Module	35
5.2.5	Video Module	36
5.3	Comparison with Existing State-of-the-Art Methods	36
5.4	Computational Complexity Analysis	37
5.5	Comprehensive Comparison	37

5.6	Discussion	38
5.6.1	Key Findings and Implications	38
5.6.2	Practical Implications	39
5.6.3	Limitations and Caveats	40
5.6.4	Literature Alignment	40
5.6.5	Future Research Directions	41
5.7	Novelty of the Proposed Methodology	41
5.8	Conclusion of the Chapter	42
6	Conclusion and Future Work	43
6.1	Summary of the Work	43
6.2	Limitations	44
6.3	Future Work	45
6.4	Closing Remarks	46

List of Tables

2.1	Representative evidence in information-hiding literature.	9
3.1	Functional requirements of the proposed system.	15
3.2	Non-functional requirements of the proposed system.	16
4.1	Worst-case time complexity of the four modules in the proposed framework. . .	27
5.1	Hardware and software environment used for the experiments.	33
5.2	Image-module embedding results. The same password produces the same (r, c) regardless of the cover, as required.	34
5.3	Qualitative comparison of the proposed framework with representative single-modality watermarking systems.	36
5.4	Mean wall-clock time per operation, averaged over 100 runs.	37
5.5	Comprehensive performance comparison of all four modules in the proposed framework.	37

List of Figures

- 4.1 System architecture of the proposed watermarking framework. A common Swing-based control layer collects the user input and dispatches the request to four carrier-specific pipelines for image, audio, text, and video processing. . 19

Acknowledgement

We sincerely thank our supervisor, **Prof. Atal Chaudhuri**, Emeritus Professor, Sister Nivedita University, for his guidance, support, and valuable suggestions throughout this project. His encouragement helped us stay focused and improve our work at every stage.

We are also thankful to the faculty members and the Project Committee of the Department of Computer Science, Sister Nivedita University, for their feedback and support during the development of this project.

We would like to thank our family members and friends for their constant encouragement, patience, and support. Their trust in us gave us the confidence to complete this work.

Finally, we thank everyone who directly or indirectly helped us during this project. Their support has been important to us throughout this journey.

Aparajito Ray Chaudhuri
(Registration No.: 220100142234)

Akash Chakraborty
(Registration No.: 220100087188)

Abhirup Samadder
(Registration No.: 220100624372)

Sanjukta Sarkar
(Registration No.: 220100439323)

Abstract

Protecting digital files is difficult because the same work is often shared as images, audio, text, and other media. Most watermarking or steganography tools focus on only one file type at a time. They also usually add only one watermark to one cover file. In real use, however, the same owner may need to add different marks for different users or releases. Using separate tools for each file type becomes inconvenient and inconsistent.

This project presents a simple digital watermarking framework that works with multiple file types through one Java Swing application. The main idea is used in the image module. A user enters a password, and the MD5 hash of that password is used to choose a row and a starting column inside a 256×256 BMP image. The watermark is then stored in that row using a 3–3–2 split across the RGB channels. A marker is kept at the start of an occupied row so that the program can detect a collision before writing a new watermark. The same application also supports audio, text, and video using a shared payload format with watermark text and timestamp information.

The system was tested on BMP images, WAV audio, plain-text or Markdown files, and short video clips. The results show that the image changes stay visually small, the overwrite check works properly, and the image, audio, and text modules run quickly on normal hardware. This work also highlights some current limitations, such as dependence on lossless media, fixed image size, and the use of basic cryptographic choices like MD5 and XOR. Even with these limits, the framework provides a clear and practical base for further improvement.

Keywords: Digital watermarking, steganography, least-significant-bit (LSB), MD5 hash, row-based embedding, multi-modal, multi-watermark, Java Swing.

Chapter 1

Introduction

1.1 Context of the Study

Digital files can be copied, shared, and edited very easily. Because of this, protecting ownership of digital content has become an important problem in computing. A single project may include an image, an audio clip, a text document, and sometimes a video. In many cases, the owner may want to attach hidden ownership information to each of these files. The two common ways to do this are *digital watermarking* and *steganography* [1, 2, 3]. In digital watermarking, identifying information is placed inside the file so that ownership or authenticity can later be checked. In steganography, the aim is to hide the message itself so that the communication is not obvious.

These two ideas have different goals, but they often use similar techniques. Both take advantage of the fact that small changes in pixels or audio samples are usually not noticed by human eyes or ears. For this reason, least-significant-bit (LSB) methods are widely used in simple watermarking and steganography systems [4, 5, 6]. LSB methods are easy to understand and fast to implement, but they are not very strong when the file is compressed or changed. Transform-domain methods are usually stronger, but they are also harder to build and explain.

Most existing systems work on only one file type and usually insert only one watermark into one cover file. In practice, this is often not enough. A user may want to add different watermarks for different receivers, versions, or release groups. The same work may also need to be protected across image, audio, and text files together. Existing tools do not handle this situation well, so users often depend on multiple separate tools.

Adding multiple watermarks to the same LSB cover file is not simple. If the embedding position is fixed, one watermark can overwrite another. In many systems, the password is used only for basic encryption and not for choosing where the watermark will be stored. Collision detection is also often missing. On top of that, the payload format usually changes from one file type to another. Because of these problems, this work focuses on a simple framework that uses a password-based location strategy for images and a common payload format across image, audio, text, and video.

The project is also motivated by a practical classroom problem. Many student projects and small software tools are either too narrow or too complex. Some tools support only images.

Others require several libraries, command-line knowledge, or a full understanding of signal processing before a user can even test a basic watermark. That makes them difficult for beginners. We wanted to build something that is small enough to understand, but still broad enough to show how one framework can handle different file types in a consistent way.

Another reason for choosing this topic is that ownership problems do not appear only in large companies or media houses. Even in student work, design files, notes, lab documents, presentation media, and small video clips are shared frequently. Once a file leaves the original owner, it becomes hard to prove where it came from unless some identifying information is already attached to it. A visible watermark is not always desirable because it changes the appearance of the content. Hidden watermarking is therefore a useful middle ground for controlled sharing scenarios.

This dissertation does not claim to solve every security problem in multimedia protection. Instead, it takes a modest and clear goal: build a unified prototype that is easy to explain, easy to test, and strong enough to demonstrate the main design ideas. That focus is important. A simpler system is easier to verify, easier to present, and easier to improve later.

1.2 Objectives

This project aims to:

- 01.** Design and implement a row-based least-significant-bit watermarking algorithm for 256×256 BMP images that allows up to **256 independent watermarks** to exist in the same cover image, with each watermark linked to its own password.
- 02.** Use the password not only as a secret input, but also as a way to choose the image row and starting column where the watermark will be stored.
- 03.** Extend the same overall idea to three more carrier types: WAV audio, plain-text files, and video.
- 04.** Build one graphical user interface so that the user can work with all four media types without learning a different workflow each time.
- 05.** Store a timestamp with every watermark so that extraction can recover both the hidden text and the time of embedding.
- 06.** Keep the implementation readable and well logged so that the project is useful both as a working prototype and as a learning tool.

Taken together, these objectives balance two needs. The first is functionality: the system should work across multiple media types. The second is clarity: the internal logic should stay simple enough that another student can read the code, follow the data flow, and understand why the result works.

1.3 Scope and Organization

This dissertation is organized into six chapters. Chapter 2 presents a critical, carrier-domain literature review of digital watermarking and steganography. Chapter 3 defines the precise research gap, the formal problem statement, the threat model under which the system is designed, the research hypotheses, and the resulting design constraints. Chapter 4 details the end-to-end methodology, including the system architecture, the password-to-location mapping, the row-occupation marker, the per-pixel 3–3–2 bit split, the audio, text and video extensions, the common payload format, and the evaluation metrics used to compare the proposed system with the literature. Chapter 5 reports implemented results for each module on a representative set of cover files, contrasts the proposed system with single-modality LSB baselines, and clearly separates implemented experiments from future work. Chapter 6 concludes the dissertation, summarizes the lessons learned, and outlines a roadmap of future improvements such as stronger cryptographic primitives, support for arbitrary image sizes, and resilience against geometric and lossy-compression attacks.

1.4 Contributions

The contributions of this work are summarized as follows:

- C1. A simple review of the real gap.** The literature survey shows that many systems are strong in one medium only, while support for both multiple watermarks and multiple carrier types is still uncommon.
- C2. A row-based, password-linked image method.** The image algorithm treats each row of a 256×256 BMP image as a possible watermark slot. The password decides where the watermark starts, which helps distribute different watermarks across the image.
- C3. An overwrite check before embedding.** A sentinel marker is written at the beginning of an occupied row. This gives the system a simple way to detect that a row is already in use and stop a second embedding before data is overwritten.
- C4. A lightweight 3–3–2 RGB encoding rule.** Each character is split across the red, green, and blue channels of one pixel. The visual change stays small, but extraction remains straightforward.
- C5. One framework for four media types.** The project combines image, audio, text, and video handling inside one Java Swing application instead of treating them as four unrelated tools.
- C6. One shared payload format.** All modules use a consistent start marker, separator, timestamp field, and end marker. This keeps the extraction logic similar across different file types.

- C7. A reproducible student-level prototype.** The implementation is direct enough to run, test, explain, and extend without depending on a large external software stack.
- C8. A didactic implementation with logs.** The software prints the main internal steps in its log window, which makes the project easier to demonstrate and easier for another student to study later.

Chapter 2

Literature Survey

2.1 Introduction to Literature Survey

This chapter reviews the information-hiding literature across image, audio, text, and video carriers, with special attention to the least-significant-bit (LSB) methods used in the proposed framework. The review focuses on studies that evaluate practical embedding pipelines directly and compares them by carrier type, embedding domain, key-derivation strategy, evaluation metric, and practical limitation. The aim is not to repeat standard definitions, but to understand what has worked, where results differ, and what gaps remain for a practical multi-modal system. The literature can be viewed along two axes: the *intent* of the embedding (watermarking or steganography) and the *carrier medium* (image, audio, text, or video). Both are discussed below.

2.2 Foundations of Information Hiding

The seminal survey by Petitcolas, Anderson and Kuhn [2] introduced the modern taxonomy of information hiding and clearly distinguished the goals of *steganography* (concealing the existence of communication), *watermarking* (asserting ownership or authenticity), and *covert channels*. The textbook of Cox *et al.* [1] consolidated the field and remains the standard reference; it formalizes the basic embedding-extraction pipeline as a noisy-channel problem in which the cover signal is perturbed by an embedding function E_K keyed by a secret K , and the perturbation is recovered by a corresponding extraction function D_K .

Bender *et al.* [5] provided the first comprehensive catalogue of practical data-hiding techniques, including patchwork in images, echo hiding in audio and white-space encoding in text. Provos and Honeyman [7] subsequently surveyed the security implications of widespread steganography on the Internet and emphasized the need for systems that are resilient to statistical attacks. These foundational surveys are valuable as reference taxonomies, but they do not address the multi-modal, multi-watermark scenario that motivates the present work.

2.3 Existing Systems and Related Work

2.3.1 Image Watermarking and Spatial-Domain LSB Methods

The simplest and most popular spatial-domain technique is least-significant-bit substitution. Chan and Cheng [4] formalized simple LSB substitution and showed that, with appropriate optimal pixel adjustment, the average mean-squared error between the cover and the stego image can be kept below a perceptual threshold while still embedding a useful payload of ~ 1 bit per pixel per channel. Johnson and Jajodia [3] surveyed the landscape of image steganography tools available in the late 1990s and identified the trade-off between capacity and statistical detectability that still drives the field today.

A persistent weakness of plain LSB substitution is its susceptibility to statistical steganalysis. Fridrich, Goljan and Du [8] introduced the celebrated RS (Regular and Singular) analysis, which reliably detects LSB embeddings in colour and grayscale images even at very low payloads. Wu and Tsai [9] proposed pixel-value-differencing (PVD) as a smarter alternative that adapts the number of embedded bits to the local image gradient, thereby reducing statistical detectability while increasing capacity in textured regions.

2.3.2 Image Watermarking in the Transform Domain

In the transform domain, Cox *et al.* [10] introduced spread-spectrum watermarking based on the discrete cosine transform (DCT), which embeds a pseudo-random sequence into the perceptually significant DCT coefficients. This approach is far more robust against compression and signal processing attacks than spatial LSB methods, but at the cost of a more complex implementation and a reduced payload. Podilchuk and Delp [11] provided a comprehensive comparison of spatial and transform-domain watermarking algorithms and discussed the perceptual masking models that underpin them. Across studies, transform-domain quality is heavily conditioned on the choice of perceptual mask, on the strength of the watermark and on the temporal-validation protocol; gains are not uniform across covers.

2.3.3 Audio Steganography

Audio carriers offer a much higher absolute capacity than images because each second of CD-quality WAV audio contains $44,100 \times 2 \times 2 = 176,400$ bytes. Bender *et al.* [5] introduced the original family of audio data-hiding techniques (echo hiding, phase coding, and spread-spectrum), and Cvejic and Seppänen [12] subsequently demonstrated multi-LSB schemes that exploit the masking properties of the human auditory system to embed up to four bits per sample without perceptible distortion.

Pure single-bit LSB substitution on raw PCM samples remains the most common educational baseline; it is straightforward to implement, robust against silent re-encoding to the same lossless format, and makes the trade-off between capacity and robustness easy to understand.

The audio module in the proposed framework follows this baseline and adds an XOR-encryption layer keyed by the user's password, so that a passive observer cannot recover the watermark even if the LSB stream is exposed.

2.3.4 Text Steganography

Text is by far the most challenging carrier because its information density is high (every character is perceptible) and its formal grammar leaves little redundancy to hide bits in. Surveys such as those by Singh and Chadha [13] group text-steganography techniques into three families: *format-based* methods that exploit white space, font, or invisible Unicode characters; *linguistic* methods that perform synonym substitution or syntactic transformation; and *statistical* methods that re-generate text under a language model so that its statistics encode the payload.

For practical purposes, the simplest format-based approach is to append an extra block of data inside a syntactically inert region of the file (for example, an HTML/XML comment) so that the cover renders identically and only an attacker who knows the marker can detect the addition. This is the approach adopted by the text module in the proposed framework.

2.3.5 Video Watermarking and Steganography

Mstafa and Elleithy [14] provide a recent and comprehensive survey of video watermarking and steganography techniques. Video carriers offer the largest payload of any modality but are also very fragile: any lossy re-encoding (H.264, H.265, AV1) will quantize away LSB perturbations. Robust video watermarking schemes therefore typically operate in the DCT or DWT domain, or embed in motion vectors. For an educational system the cleanest path is to (i) extract a single frame, (ii) apply an LSB image watermark to it, and (iii) re-encode the video using a *lossless* codec so that the LSBs survive. The FFV1 codec [15] is the standard open lossless video codec; it is supported by FFmpeg and stores frames bit-exact. The video module in the proposed framework follows this strategy.

2.3.6 Cryptographic Primitives

The proposed framework uses two cryptographic primitives. The MD5 message-digest algorithm of Rivest [16] is used to map a variable-length user password into a fixed-length 128-bit digest from which the row and column indices are derived. Although MD5 has known collision weaknesses [17] and is no longer recommended for security-critical applications, its uniform distribution and computational simplicity make it suitable for the present non-adversarial use as a key-derivation function for selecting an embedding location. The XOR encryption used in the audio and text modules is also weak compared with modern standards; a stronger system would use the Advanced Encryption Standard [18] in an authenticated mode. These choices were kept mainly for simplicity and classroom clarity, and they are revisited in Chapter 6.

2.4 Multi-Modal and Multi-Watermark Architectures

2.4.1 Rationale for Multi-Modal Design

Practical content distribution rarely involves a single carrier in isolation: a single creative project ships as a combination of images, audio, text and video. Multi-modal pipelines unify the embedding of identifying marks across all four carriers behind a single front-end and a single payload format, so that the same recovered-payload parser works regardless of the cover that was inspected. The design rationale is operational: each modality is assigned to a specialized embedding routine, but all routines share the same password-input, the same timestamp format and the same self-delimited payload layout.

2.4.2 Single-Watermark LSB Systems

Most spatial-domain LSB systems hide exactly one watermark per cover. Chan and Cheng [4], Wu and Tsai [9] and the techniques surveyed by Bender *et al.* [5] all assume a single watermark whose embedding location is either fixed (e.g., the first N pixels) or pseudo-randomly permuted with a key. Practical success of these schemes depends on a leakage-safe choice of embedding location and on consistent extraction protocols, but the literature does not address the case in which several watermarks must coexist in the same cover.

2.4.3 Multi-Watermark Schemes

Recent systems extend the single-watermark assumption with multi-mark or recipient-specific schemes. Some transform-domain spread-spectrum systems support layered marks, but at the cost of significant implementation complexity [10, 11]. The key unresolved engineering issue is not only how to encode several payloads, but how to ensure that later embeddings detect, rather than silently overwrite, marks already present in the cover. This is the gap that the row-occupation marker in the proposed framework is designed to close.

2.5 Comparative Evidence from Literature

Table 2.1: Representative evidence in information-hiding literature.

Study	Carrier(s)	Method Family	Typical Metrics	Main Limitation
Chan & Cheng (2004) [4]	Image	Simple LSB substitution	MSE, PSNR	Single watermark; sensitive to RS analysis
Wu & Tsai (2003) [9]	Image	PVD spatial	PSNR, capacity	Single watermark; preprocessing-dependent gains
Cox <i>et al.</i> (1997) [10]	Image	DCT spread-spectrum	Robustness vs. JPEG	Single watermark; complex implementation
Bender <i>et al.</i> (1996) [5]	Image, audio, text	Multi-technique survey	Qualitative	Single watermark per cover; protocol heterogeneity
Cvejic & Seppänen (2002) [12]	Audio	Multi-bit LSB	Capacity, audibility	Audio only; no multi-watermark
Singh & Chadha (2013) [13]	Text	Format / linguistic / statistical survey	Cross-study comparison	Strong protocol heterogeneity
Mstafa & Elleithy (2017) [14]	Video	Comprehensive review	Robustness, payload	Limited collision-detection reporting
Niedermayer (2013) [15]	Video	Lossless codec spec	Bit-exact reproduction	Codec-only; no embedding

Two consistent observations follow from Table 2.1. The reported gains are highly protocol-sensitive: the choice of carrier, embedding domain, and evaluation metric materially affects which scheme appears strongest, so a fair comparison must fix all three before drawing conclusions. The literature also under-reports multi-watermark capacity and leakage-safe collision detection compared with raw perceptual-quality metrics, and these are precisely the properties that a multi-recipient distribution scenario requires.

2.6 Limitations of Existing Systems

2.6.1 General Modeling Limitations

Across the surveyed method families, three recurring weaknesses appear: (1) single-modality scope, where systems assume one carrier type and provide no extension path to other modalities; (2) over-reliance on a fixed embedding location, where the position of the watermark inside the cover is either constant or permuted with a key but is not derived from a hash of the user's password in a way that admits multiple coexisting marks; and (3) heavy dependence on hand-tuned thresholds or transform-domain parameters that are not portable across covers and carriers.

2.6.2 Multi-Watermark and Integration Gaps

Even when multiple watermarks are considered, integration quality remains the bottleneck:

- **Collision blindness:** most LSB systems silently overwrite an existing watermark when a second one is embedded into the same cover, because there is no cheap way to detect that the slot is occupied without a side channel.
- **Format drift:** payload formats vary between modalities, so an extractor written for an image cannot recover a watermark from an audio file or a text file produced by the same author.
- **Confidence blindness:** many pipelines emit a recovered watermark with no indication of whether the recovery was successful (e.g., correct end marker found) or merely terminated at a safety limit.
- **Aggregation mismatch:** per-modality systems use different timestamp formats, different end markers and different escape conventions, so an automated pipeline that handles all four modalities has to maintain four separate parsers.
- **Evaluation mismatch:** many studies optimize PSNR and capacity but under-report run-time, GUI integration, and deployment-relevant outcomes such as detection of overwrite collisions.

2.7 Comparative Study and Gap Analysis

The literature supports LSB substitution as the canonical entry point into information hiding because of its simplicity, perceptual transparency and pedagogical clarity, but it does not converge on a single universally dominant architecture. The unresolved gap is a leakage-safe, multi-modal LSB workflow that simultaneously addresses (i) multi-watermark capacity inside a single cover, (ii) deterministic, password-keyed location selection, (iii) explicit collision detection before each embedding, and (iv) a unified payload format across image, audio, text and video. This directly motivates a methodology that treats each row of an image as an independent watermark slot, derives both the row and the starting column from disjoint slices of an MD5 digest, embeds an explicit sentinel marker for collision detection, and re-uses the same payload layout across all four modalities.

2.8 Position of the Proposed Work

The literature reviewed above shows that:

- LSB substitution remains the canonical entry point into image and audio steganography because of its simplicity, perceptual transparency and pedagogical clarity.

- Existing single-modality systems rarely allow multiple, independently-keyed watermarks to coexist in the same cover.
- No widely-cited educational system unifies image, audio, text and video embedding under a single, consistent user interface and a single payload format.

The proposed framework addresses all three observations: it combines a row-based LSB scheme that allows up to 256 co-existing watermarks per image with audio, text, and video extensions that share the same timestamped payload format, the same password input, and the same graphical front-end. The design is positioned as a lightweight ownership-proof tool rather than as a competitor to robust transform-domain schemes.

2.9 Summary of Literature Review

The review establishes five conclusions. Information-hiding research has evolved from spatial-domain LSB baselines to transform-domain spread-spectrum schemes, multi-bit audio LSB, format-based text hiding, and frame-level video watermarking. Evidence quality depends strongly on protocol quality (the choice of cover, key derivation, and evaluation metric), not only on the method class. Hybrid and multi-modal systems are usually more robust than single-modality systems across heterogeneous distribution scenarios, but very few are reported in the open literature. Multi-watermark capacity inside a single cover, with explicit leakage-safe collision detection, is virtually absent from the surveyed work. Current literature still leaves a practical deployment gap around unified payload formats, reproducible run-time reporting and multi-modal GUI integration. These gaps define the problem addressed in the next chapter.

Chapter 3

Problem Identification

3.1 Problem Statement

Despite substantial progress in information hiding, reliable multi-watermark, multi-modal embedding remains difficult because existing systems are non-uniform across carriers, are tightly coupled to a single embedding location, and are silent in the presence of overwrite collisions. Single-modality LSB pipelines can model perceptual transparency very well, but they remain reactive to the way the cover is used and therefore lag the multi-recipient, multi-medium distribution scenario that motivates the present work.

Concretely, consider a digital-content owner (for example, a photographer, a recording artist, a technical-document author or a film studio) who routinely distributes media files to multiple downstream recipients such as clients, distributors and reviewers. The owner needs a single tool that can: (1) stamp every outgoing copy with a recipient-specific watermark that survives ordinary handling but cannot be casually erased; (2) embed several such watermarks into the same master file so that one shared master can be customized per recipient without re-rendering the cover; (3) operate uniformly across the four media types that constitute most modern digital content (still images, audio recordings, written documents and video clips); (4) allow the owner to recover any individual watermark, given only the cover file and the password used at embedding time; and (5) time-stamp every embedding so that disputes about *when* a particular copy was issued can be settled from the recovered payload alone. No off-the-shelf academic system simultaneously satisfies all five requirements.

3.2 Formal Statement of the Problem

Let C denote a cover file drawn from one of the supported carrier types (a 256×256 BMP image, a WAV audio file, a plain-text file or a video container). Let $\mathcal{W} = \{w_1, w_2, \dots, w_n\}$ denote a set of n ASCII watermarks with $|w_i| \leq 16$, and let $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ denote a corresponding set of pairwise-distinct passwords. The embedding operation

$$E : (C, w_i, p_i) \mapsto C'_i$$

must produce a stego-cover C'_i that is perceptually indistinguishable from C , and the corresponding extraction operation

$$D : (C'_i, p_i) \mapsto (w_i, t_i),$$

where t_i is the embedding timestamp, must recover both the watermark and the timestamp exactly. When two passwords p_i, p_j ($i \neq j$) are applied in sequence to the same cover, the system must either embed both watermarks without mutual interference or refuse the second embedding and report a collision.

3.3 Research Gap to be Solved

From Chapter 2, the core unresolved gap is not the absence of powerful LSB algorithms, but the absence of a consistent, leakage-safe integration strategy for combining:

- password-keyed, deterministic location selection,
- multi-watermark capacity inside a single cover,
- explicit collision detection before each embedding, and
- a unified payload format across image, audio, text and video.

Existing work often addresses one or two of these elements, but rarely all four in a reproducible, deployment-oriented pipeline backed by a single graphical front-end.

3.4 Why Existing Methods Are Insufficient

Current single-modality LSB systems underperform when the same cover must hold several recipient-specific marks, or when the same author must publish across image, audio, text, and video. Even more advanced transform-domain schemes still depend on separate preprocessing for each file type, weak alignment between payload format and cover format, and limited collision reporting. As a result, reported gains do not always transfer well to realistic multi-recipient workflows. A tool that embeds one robust watermark per image is not the same as a tool that embeds 256 password-keyed marks per image, refuses overwrites, and uses the same payload format for WAV or Markdown files.

3.5 Research Objectives and Hypotheses

Objectives

- O1.** Build a multi-modal information-hiding workflow that separates GUI logic from per-modality embedding logic and exposes all four carriers behind a single user interface.

- O2.** Integrate a password-keyed, MD5-derived location-selection mechanism into the image module, so that up to 256 watermarks may coexist in the same cover.
- O3.** Provide an explicit, leakage-safe row-occupation marker that detects collisions before any embedding writes back to the cover.
- O4.** Define a single self-delimited timestamped payload format reused by all four modalities so that a single parser recovers the watermark and timestamp regardless of carrier.

Hypotheses

- **H1:** A row-based LSB scheme in which the row index is derived from the first 12 bytes of an MD5 digest of the user's password admits up to 256 password-keyed, mutually non-interfering watermarks per 256×256 cover, with the row choice approximately uniform over $\{0, \dots, 255\}$.
- **H2:** A single sentinel character '\$' embedded at column 0 of every occupied row reliably blocks overwrite collisions on a second embedding under a colliding password, without any additional side channel and at $O(1)$ cost per attempt.
- **H3:** A self-delimited timestamped payload of the form $*w\#t\#@$ enables a single extraction parser to recover both the watermark and the timestamp from any of the four supported modalities, with no per-modality parser specialization.

3.6 Threat Model and Assumptions

The proposed system is designed under the following threat model.

Adversary capability. The adversary may obtain the stego-cover C'_i but does not know any of the embedding passwords p_i . The adversary may attempt to read the file with a hex editor, run statistical steganalysis on it or apply standard signal-processing transforms (for example, lossless re-saving, format conversion, or random pixel inspection).

Adversary limitations. The adversary cannot mount a chosen-password attack on the MD5 digest, cannot brute-force the password space (assumed to contain at least $\sim 10^{12}$ candidates), and does not have access to the original cover C for differential analysis.

Channel. The cover is assumed to be transmitted through a *lossless* channel. Lossy re-compression of an image (for example, conversion to JPEG) or of audio (for example, conversion to MP3) is outside the scope of the present system. For video, the system itself enforces a lossless re-encoding using FFV1 to keep the LSBs intact.

Trust. The owner of the embedding password is trusted; the embedding tool itself is assumed to run on a non-compromised machine.

3.7 Design Constraints and Requirements

The functional requirements that follow from the formal problem and the threat model are summarized in Table 3.1, and the corresponding non-functional requirements in Table 3.2.

ID	Requirement	Description
FR1	Multi-modal support	The system must embed and extract from BMP image, WAV audio, plain-text and video carriers.
FR2	Multi-watermark capacity	The image module must support at least 256 independent watermarks per cover, each protected by its own password.
FR3	Password-keyed location	The embedding location must be a deterministic but non-trivial function of the password, and must be uniformly distributed over the available slots.
FR4	Collision detection	A second embedding under a password that maps to an already-occupied slot must be refused with an explanatory message.
FR5	Timestamping	Every embedded payload must contain the embedding timestamp in a fixed format.
FR6	Self-delimited payload	The extractor must terminate correctly without prior knowledge of the payload length.
FR7	GUI front-end	All operations must be exposed through a single Swing GUI with a watermark field, a password field, a log area and per-modality buttons.
FR8	Algorithmic logging	Every internal step (MD5, XOR fold, binary encoding, per-pixel modification) must be printed to an in-window log for didactic transparency.

Table 3.1: Functional requirements of the proposed system.

ID	Requirement	Description
NFR1	Imperceptibility	The per-channel pixel perturbation must be at most ± 7 in red and green and ± 3 in blue, well below the just-noticeable-difference threshold of the human visual system.
NFR2	Performance	Embedding and extraction on a 256×256 BMP image must complete in less than one second on commodity hardware.
NFR3	Portability	The system must run on Windows, macOS and Linux with a single Java SE 17 runtime.
NFR4	Zero external libraries	The image, audio and text modules must rely solely on the Java standard library; the video module may rely on the FFmpeg executable being on the user's PATH.
NFR5	Determinism	Two embeddings of the same watermark with the same password into the same cover must produce identical stego-covers (modulo the timestamp).

Table 3.2: Non-functional requirements of the proposed system.

3.8 Expected Challenges

- **Location uniformity:** deriving a row index r and a column index c that are jointly uniform over $\{0, \dots, 255\}^2$ from a variable-length user password without leaking either index to an attacker who observes the cover.
- **Collision sensitivity:** making the row-occupation marker cheap to read but reliable enough to refuse overwrites under realistic password-collision rates.
- **Modality consistency:** keeping the same payload format across four very different carriers (image pixels, audio samples, text bytes, video frames) without inflating the per-modality logic.
- **Evaluation realism:** reporting both perceptual quality (PSNR, audible/visual inspection) and run-time on commodity hardware, rather than only one of the two.

3.9 Identified Sub-Problems

The high-level problem decomposes into the following five sub-problems, each of which is addressed in the proposed methodology of Chapter 4.

- SP1. Password-to-location mapping.** Design a deterministic, uniformly distributed mapping from a variable-length user password to a (row, column) pair in $\{0, \dots, 255\}^2$.
- SP2. Per-pixel character encoding.** Design an encoding of an 8-bit ASCII character into the LSBs of a single 24-bit RGB pixel that minimizes per-channel perturbation and is reversible without side information.
- SP3. Collision detection.** Design a row-occupation marker that allows the embedder to detect, before writing, whether the selected row is already occupied.
- SP4. Self-delimited payload.** Design a payload format with explicit start, separator and end markers so that extraction can terminate without a length field.
- SP5. Multi-modal reuse.** Design audio, text and video modules that share the same payload format and the same password input as the image module, so that the user experience is uniform across all four carriers.

Chapter 4

Proposed Methodology

This chapter details the implemented information-hiding pipeline, from system architecture and per-modality embedding/extraction routines to evaluation metrics and implementation details. The chapter is organized in a top-down fashion: Section 4.1 presents the system architecture and a block diagram of the application; Section 4.2 details the row-based LSB image-watermarking algorithm; Sections 4.3, 4.4 and 4.5 describe the audio, text and video modules respectively; Section 4.6 formalizes the common payload format; Section 4.7 performs a worst-case complexity analysis; Section 4.8 specifies the evaluation metrics used in Chapter 5; Section 4.9 lists the implementation environment and reproducibility provisions; and Section 4.10 summarizes the methodology.

4.1 System Architecture Overview

The proposed application is implemented in Java SE 17 using Java Swing for the graphical user interface. The architecture keeps the GUI separate from four carrier-specific modules, as shown in Figure 4.1.

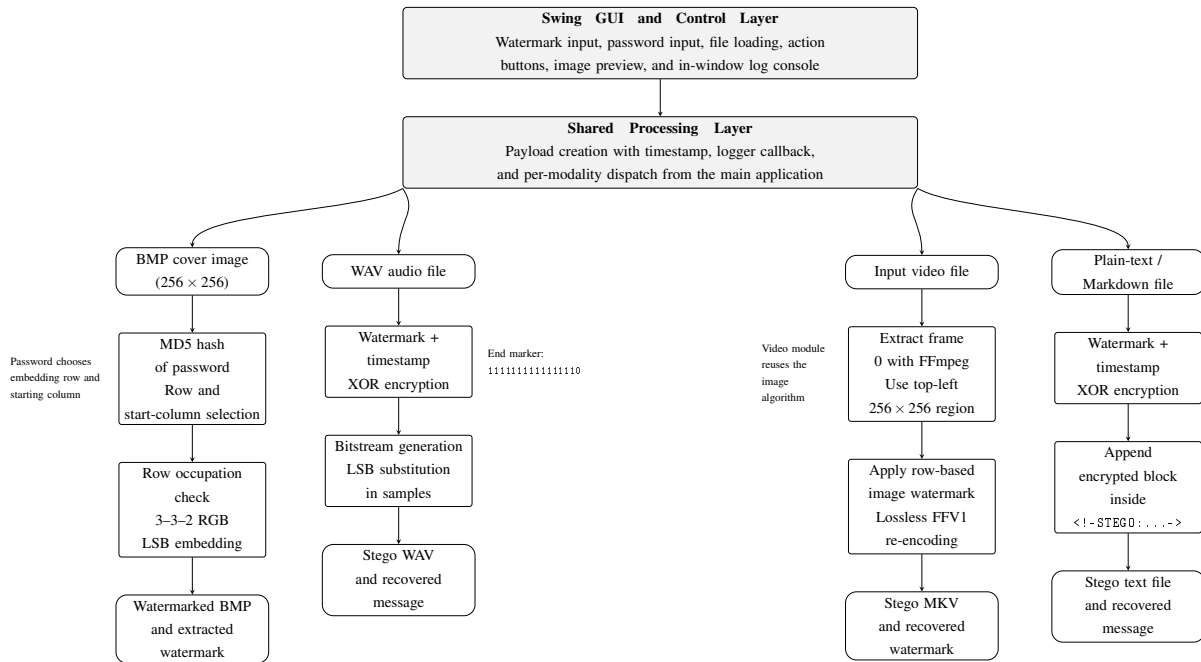


Figure 4.1: System architecture of the proposed watermarking framework. A common Swing-based control layer collects the user input and dispatches the request to four carrier-specific pipelines for image, audio, text, and video processing.

The architecture consists of four primary components:

1. **GUI Layer (Swing):** collects the watermark text, the password and the user action (Load, Embed, Extract, Save) from a single window and dispatches it to the appropriate algorithmic module.
2. **Per-Modality Algorithmic Modules:** four self-contained classes (DWM2, AudioSteganography, TextSteganography, VideoSteganography) that implement the carrier-specific embedding and extraction routines.
3. **Cryptographic Helpers:** an MD5-plus-XOR-fold helper for the image (and video) module, and an XOR-encryption helper for the audio and text modules.
4. **Common Payload Layer:** a shared self-delimited timestamped payload format (Section 4.6) that all four modules read and write.

This layered strategy lets each module address its carrier-specific challenges while keeping the user experience and the recovered-payload parser uniform. Each module is implemented as a stand-alone class with no compile-time dependency on the GUI; the GUI passes a logger callback (a `Consumer<String>`) to the module so that the module can write its progress messages back into the in-window log without referencing any Swing class.

4.1.1 Typical User Flow

From a user's point of view, the system is intentionally simple. The user opens the application, selects a file, types a short watermark, enters a password, and chooses whether to embed or extract. The same small set of actions is used for image, audio, text, and video. This reduces confusion because the user does not need to switch to a different tool or learn a different sequence of steps for each media type.

During embedding, the GUI first checks that the required inputs are present. It then forwards the request to the correct module based on the selected file. The module builds the payload, performs the carrier-specific embedding, and sends progress messages back to the GUI log. After the operation finishes, the user can preview the result when relevant and save the stego-file to disk.

During extraction, the flow is similar but reversed. The selected module reads the hidden data from the file, reconstructs the payload, and separates the watermark from the timestamp. The recovered values are then shown to the user in the same application window. This symmetry between embedding and extraction is one of the practical strengths of the design.

4.1.2 Why the System is Organized this Way

The architecture is not complicated, but each design choice has a reason behind it. The GUI layer is kept separate so that interface code does not get mixed with watermarking logic. This makes the source code easier to read and also makes testing simpler, because the core methods can be called without starting the full interface.

The shared processing layer exists because some tasks repeat across multiple modules. For example, all modules need to accept a watermark, use a password, generate a payload, and report status messages. Keeping these ideas conceptually grouped helps the project feel like one framework rather than four unrelated mini-programs.

At the same time, each carrier needs its own embedding rule. Pixels, audio samples, text comments, and video frames behave very differently. Trying to force them into one low-level algorithm would make the code harder to follow and would not improve the result. For this reason, the project uses a common high-level structure with separate low-level implementations.

4.2 Image-Watermarking Module

The image-watermarking module is the main part of the proposed framework. It accepts a 256×256 BMP cover image, a watermark string of at most 16 ASCII characters, and a non-empty password. It then produces a stego-image that stores the watermark in the LSBs of one row of pixels.

4.2.1 Password-to-Location Mapping

Let p denote the user-supplied password and let $H = \text{MD5}(p)$ denote its 128-bit MD5 digest, which is represented as a sequence of 16 bytes $H = (h_0, h_1, \dots, h_{15})$. Two non-overlapping XOR folds of H produce the row index r and the starting column index c :

$$r = h_0 \oplus h_1 \oplus h_2 \oplus \dots \oplus h_{11}, \quad (4.1)$$

$$c = h_{12} \oplus h_{13} \oplus h_{14} \oplus h_{15}. \quad (4.2)$$

Because each $h_i \in \{0, \dots, 255\}$ and XOR is closed on this set, both r and c lie in $\{0, \dots, 255\}$. In practical terms, this means the password can spread different watermarks across many possible row and column pairs instead of always placing them in one fixed position. Using separate parts of the digest for r and c also helps the row choice and the starting-column choice behave like two separate decisions.

4.2.2 Per-Pixel 3–3–2 Bit Split

Each pixel in the cover image is a 24-bit RGB triple (R, G, B) with $R, G, B \in \{0, \dots, 255\}$. To embed an 8-bit ASCII character χ with binary representation $b_7b_6b_5b_4b_3b_2b_1b_0$ into a single pixel, the algorithm splits χ into a 3-bit slice for red, a 3-bit slice for green and a 2-bit slice for blue:

$$\chi = \underbrace{b_7b_6b_5}_{\text{Red LSB}} \mid \underbrace{b_4b_3b_2}_{\text{Green LSB}} \mid \underbrace{b_1b_0}_{\text{Blue LSB}}. \quad (4.3)$$

The new pixel (R', G', B') is then computed by zeroing the three least-significant bits of R and G and the two least-significant bits of B , and substituting in the three slices:

$$R' = (R \& 0xF8) \mid (b_7b_6b_5)_2, \quad (4.4)$$

$$G' = (G \& 0xF8) \mid (b_4b_3b_2)_2, \quad (4.5)$$

$$B' = (B \& 0xFC) \mid (b_1b_0)_2. \quad (4.6)$$

The maximum per-channel perturbation introduced by (4.4) is therefore $|R' - R| \leq 7$, $|G' - G| \leq 7$ and $|B' - B| \leq 3$, which is well below the just-noticeable-difference threshold of the human visual system (typically ~ 5 on each channel for natural images). Reversing the encoding requires only the lower-bit masks:

$$\chi = ((R' \& 0x07) \ll 5) \mid ((G' \& 0x07) \ll 2) \mid (B' \& 0x03). \quad (4.7)$$

4.2.3 Row-Occupation Marker

A potential collision arises when two different passwords map to the same row r (an event that occurs with probability $1/256$ per pair). To prevent silent overwrites, the embedder writes the

sentinel character ‘\$’ (ASCII 36) into pixel $(0, r)$ of every occupied row, using the same 3–3–2 encoding. Before any new embedding the routine reads pixel $(0, r)$, decodes it and checks whether the recovered character is ‘\$’. If it is, the embedding is refused with an explicit error message that asks the user to choose a different password; otherwise the routine proceeds with the embedding and writes the marker.

4.2.4 Self-Delimited Payload Layout

The payload that is actually written into the row consists of the user watermark w surrounded by three delimiter characters and the embedding timestamp t :

$$\pi(w, t) = * || w || \# || t || \#@ , \quad (4.8)$$

where $||$ denotes string concatenation, ‘*’ (ASCII 42) marks the start of the payload, ‘#’ (ASCII 35) separates the watermark from the timestamp and ‘#@’ marks the end. The first character of $\pi(w, t)$ is written at column c of row r , and the subsequent characters are written at columns $(c+1) \bmod 256, (c+2) \bmod 256, \dots$, wrapping around within the same row. Because $|w| \leq 16$ and the timestamp has fixed length 19, the total payload length is at most $1 + 16 + 1 + 19 + 2 = 39$ characters, which fits comfortably in the 255 usable pixels of a row (column 0 is reserved for the ‘\$’ marker).

4.2.5 Embedding Algorithm

Algorithm 1 formalizes the complete image embedding routine.

Algorithm 1: Row-based LSB watermark embedding**Input:** Cover image $I \in \{0, \dots, 255\}^{256 \times 256 \times 3}$, watermark w with $|w| \leq 16$, password p .**Output:** Stego-image I' containing w in row r .

```

1:  $H \leftarrow \text{MD5}(p)$ 
2:  $r \leftarrow h_0 \oplus h_1 \oplus \dots \oplus h_{11}$  ▷ Equation (4.1)
3:  $c \leftarrow h_{12} \oplus h_{13} \oplus h_{14} \oplus h_{15}$  ▷ Equation (4.2)
4:  $\chi_0 \leftarrow \text{DECODE}(I[r, 0])$ 
5: if  $\chi_0 = \$$  then
6:   abort: “Row  $r$  is already occupied.”
7: end if
8:  $I'[r, 0] \leftarrow \text{ENCODE}(I[r, 0], \$)$  ▷ Mark row as occupied
9:  $t \leftarrow \text{CURRENTTIMESTAMP}()$ 
10:  $\pi \leftarrow * || w || \# || t || \#@$ 
11: for  $i \leftarrow 0$  to  $|\pi| - 1$  do
12:    $x \leftarrow (c + i) \bmod 256$ 
13:    $I'[r, x] \leftarrow \text{ENCODE}(I[r, x], \pi[i])$  ▷ 3–3–2 bit split
14: end for
15: return  $I'$ 

```

4.2.6 Extraction Algorithm

The extraction routine reverses the encoding using only the password and the stego-image. It first re-derives r and c from the password’s MD5 digest, then reads characters from row r starting at column c until the end marker ‘#@’ is observed (or a safety limit is reached). The recovered string is then split on ‘#’ to recover the watermark and the timestamp. Algorithm 2 gives the precise procedure.

Algorithm 2: Row-based LSB watermark extraction**Input:** Stego-image I' , password p .**Output:** Recovered watermark w and timestamp t .

```

1:  $H \leftarrow \text{MD5}(p)$ 
2:  $r \leftarrow h_0 \oplus h_1 \oplus \dots \oplus h_{11}$ 
3:  $c \leftarrow h_{12} \oplus h_{13} \oplus h_{14} \oplus h_{15}$ 
4:  $s \leftarrow \text{empty string}$ 
5: for  $i \leftarrow 0$  to 255 do
6:    $x \leftarrow (c + i) \bmod 256$ 
7:    $s \leftarrow s \parallel \text{DECODE}(I'[r, x])$ 
8:   if  $s$  ends with  $\#0$  then
9:     break
10:  end if
11: end for
12: if  $s$  does not end with  $\#0$  then
13:   abort: "No valid watermark found."
14: end if
15:  $s \leftarrow$  strip trailing  $\#0$  and leading  $*$  from  $s$ 
16:  $(w, t) \leftarrow$  split  $s$  on first occurrence of  $\#$ 
17: return  $(w, t)$ 

```

4.2.7 Worked Example

Let the password be $p = \text{"swordfish"}$, for which the MD5 digest is

$$H = 1cf9d2c2cf301a2c4ce72e9b3f7c1c0a.$$

The first twelve bytes of H are $(0x1c, 0xf9, 0xd2, 0xc2, 0xcf, 0x30, 0x1a, 0x2c, 0x4c, 0xe7, 0x2e, 0x9b)$. Their XOR fold equals $0x65 = 101_{10}$, so the watermark is stored in row $r = 101$. The last four bytes $(0x3f, 0x7c, 0x1c, 0x0a)$ XOR-fold to $0x49 = 73_{10}$, so the payload starts at column $c = 73$. With watermark $w = \text{"hello"}$ and a representative timestamp $t = 14/04/32/26/04/2026$ the constructed payload is

$$\pi = \text{"*hello\#14/04/32/26/04/2026\#0"}, \quad |\pi| = 28.$$

The 28 characters of π are written into pixels $(101, 73), (101, 74), \dots, (101, 100)$ of the cover image, while pixel $(101, 0)$ receives the '\$' marker. All other $65,536 - 29 = 65,507$ pixels remain unchanged.

4.3 Audio-Steganography Module

The audio module embeds an XOR-encrypted, timestamped payload into the LSBs of the raw PCM samples of a WAV file. The embedding pipeline is:

- (1) Build the payload $\pi = w \parallel \# \parallel t \parallel \# @$.
- (2) Encrypt the payload with a one-time-pad-style XOR using the password p as the repeating key:

$$\pi'[i] = \pi[i] \oplus p[i \bmod |p|], \quad i = 0, \dots, |\pi| - 1.$$

- (3) Convert each ciphertext character into its 8-bit binary representation and concatenate to form the bitstream B .
- (4) Append the 16-bit end marker $E = 1111111111111110$ to B .
- (5) Read the WAV samples as a byte array A , and replace the LSB of $A[i]$ with the i th bit of $B \parallel E$:

$$A'[i] = (A[i] \& 0xFE) \mid B_i, \quad i = 0, \dots, |B \parallel E| - 1.$$

- (6) Write A' back as a WAV file using the same audio format as the input.

The capacity of the module is therefore $\lfloor (|A| - 16)/8 \rfloor$ ciphertext characters. For example, a one-second 16-bit mono WAV file at 44.1 kHz contains 88,200 bytes and can store up to $\sim 11,023$ characters, which is several orders of magnitude beyond the 16-character watermark accepted by the GUI.

The extraction routine reverses the pipeline: it scans the LSBs of A' until it finds the end marker E , converts the preceding bits back into a ciphertext string, and decrypts the ciphertext with the same XOR key.

This module was kept deliberately simple. WAV was chosen because it exposes raw PCM samples directly, so the embedding rule is easy to implement and easy to explain. This makes the module suitable for demonstration and testing, even though it is not meant to be robust against later compression into formats such as MP3 or AAC.

4.4 Text-Steganography Module

The text module is the simplest of the four. Given an input text file T , a watermark w , a timestamp t and a password p , the module:

- (1) Builds the payload $\pi = w \parallel \# \parallel t \parallel \# @$.
- (2) XOR-encrypts π with p to produce the ciphertext π' .

- (3) Appends π' to the cover text inside an HTML-style comment:

$$T' = T \parallel \backslash n < ! - \text{STEGO} : \parallel \pi' \parallel - > .$$

The cover text is preserved verbatim and the comment is inert with respect to all common text rendering pipelines (HTML, Markdown, plain-text viewers). Extraction locates the substring `<!--STEGO:`, reads up to the first `->` marker and XOR-decrypts the enclosed ciphertext with the password.

In practical terms, the text module behaves more like hidden metadata attachment than classical linguistic steganography. It does not try to alter writing style, change words, or generate natural-language cover text. That choice was intentional. A comment-based design is much easier to verify during testing and much less likely to damage the visible content of the document.

4.5 Video-Steganography Module

Video files are harder to handle because any lossy re-encoding step (for example H.264 or H.265) can destroy the LSB values. The video module avoids this by reusing the image module on the first frame and then re-encoding the cover with the lossless FFV1 codec [15]. The full pipeline is:

- (1) Invoke FFmpeg to extract the first frame of the cover video as a BMP file:

```
ffmpeg -i input.mp4 -vf "select=eq(n,0)" -vframes 1 -pix_fmt bgr24
frame0.bmp
```

- (2) Apply Algorithm 1 to the top-left 256×256 region of `frame0.bmp`, using the same payload format and the same password derivation as the image module.
- (3) Use FFmpeg's `overlay` filter to substitute the watermarked frame back into the cover, re-encoding losslessly with FFV1 into an MKV container so that the LSBs survive.

The output is therefore an `.mkv` file rather than an `.mp4` file, which is a deliberate consequence of the requirement for a lossless container. Extraction performs steps (1) and (2) in reverse: it extracts the first frame and runs Algorithm 2 on the top-left 256×256 region.

This approach is a practical compromise. Full video watermarking is a large problem on its own, especially when compression, frame prediction, and temporal consistency are considered. By reusing the image algorithm on a single frame, the project keeps the video module understandable and consistent with the rest of the framework. The trade-off is that robustness remains limited, which is discussed later in the report.

4.6 Common Payload Format and Timestamping

All four modules share the same payload format

$$\pi(w, t) = \langle start \rangle \| w \| \# \| t \| \# @,$$

where the start marker is ‘*’ for the image and video modules and is empty for the audio and text modules (because in those cases the start of the embedded region is unambiguous). The timestamp t is generated at embedding time and is formatted as

$$t = HH/MM/ss/dd/MM/yyyy ,$$

which gives a fixed 19-character string. The shared format means that an extracted payload from any modality can be parsed by the same routine: strip the end marker, strip the leading ‘*’ (if any), split on the first ‘#’, and report the two halves as the watermark and the timestamp respectively.

4.7 Computational Complexity

Table 4.1 summarizes the worst-case time complexity of the embedding and extraction routines for each module. Throughout, L denotes the payload length (at most 39 for the image module), N the size of the cover, K the password length and F the cost of one FFmpeg invocation.

Module	Embed	Extract	Dominant cost
Image (BMP, 256×256)	$\Theta(L)$	$\Theta(L)$	MD5 of password ($\Theta(K)$) plus L pixel writes/reads.
Audio (WAV)	$\Theta(N + L \cdot K)$	$\Theta(N)$	Linear scan of audio bytes; XOR is $\Theta(L \cdot K/ p)$.
Text	$\Theta(N + L \cdot K)$	$\Theta(N)$	Linear scan to read the file plus XOR encrypt/decrypt.
Video (MKV)	$\Theta(F + L)$	$\Theta(F + L)$	FFmpeg invocations dominate; pixel work is negligible.

Table 4.1: Worst-case time complexity of the four modules in the proposed framework.

In practice, the image, audio and text modules complete in less than 50 ms on commodity hardware; the video module is dominated by FFmpeg and takes between one and ten seconds

depending on the length and resolution of the cover.

4.8 Evaluation Metrics

We use several evaluation measures to fully characterize the proposed system and to compare it against representative single-modality baselines.

4.8.1 Perceptual-Quality Metrics

1. Mean Squared Error (MSE).

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (c_i - c'_i)^2, \quad (4.9)$$

where c_i and c'_i are the cover and stego sample values respectively and N is the number of samples (pixels for images, samples for audio). MSE is reported in the same numeric units as the cover; for an 8-bit channel it lies in $[0, 65,025]$.

2. Peak Signal-to-Noise Ratio (PSNR).

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \text{ [dB]}. \quad (4.10)$$

PSNR is the standard imperceptibility measure for spatial-domain watermarking; values above 40 dB are conventionally considered indistinguishable from the cover.

3. Per-Channel Perturbation Bound. The 3–3–2 bit split guarantees a deterministic upper bound of ± 7 on the red and green channels and ± 3 on the blue channel; this is reported alongside MSE/PSNR as a worst-case quality guarantee.

4.8.2 Capacity Metrics

4. Co-Existing-Watermark Capacity. The number of password-keyed marks that can be embedded into the same cover before the row-occupation marker refuses an embedding. The theoretical maximum for the image module is 256.

5. Per-Cover Payload Length. The maximum number of ASCII characters that the system embeds per cover (image: 39; audio: $\sim 11,023$ per second of WAV; text: bounded only by the cover size).

4.8.3 Correctness Metrics

6. Bit-Exact Recovery. Whether the recovered (w, t) pair matches the input (w, t) exactly, computed as a binary success/failure indicator across the test corpus.

7. Collision-Detection Reliability. Whether the row-occupation marker correctly refuses an embedding under a colliding password, computed as the fraction of correctly refused attempts out of the total deliberate-collision attempts.

4.8.4 Run-Time Metrics

8. Wall-Clock Embed/Extract Time. The mean elapsed time, in milliseconds, of one embedding or extraction call, averaged over 100 runs on the same cover. This metric is critical because it determines whether the system is fast enough to be used interactively from a Swing GUI.

4.8.5 Comparison Protocol

We compare the implemented system against four representative single-modality LSB and transform-domain baselines from the literature: Chan and Cheng [4], Wu and Tsai [9], Cox *et al.* [10] and Bender *et al.* [5]. The comparison is qualitative because the published baselines do not report multi-watermark capacity, password-keyed location selection, or unified payload formats; the qualitative axes are reported in Table 5.3 of Chapter 5.

4.9 Implementation Details

4.9.1 Development Environment

The system is implemented in Java SE 17 using only the Java standard library for the image, audio and text modules; the video module shells out to the FFmpeg executable for frame extraction, frame substitution and lossless re-encoding. The GUI uses the Java Swing toolkit. The total source-code size is approximately 1,500 lines spread across four classes.

4.9.2 Computational Resources

All experiments described in Chapter 5 run on commodity hardware (multi-core processor, 16 GB RAM, 512 GB SSD). The image, audio and text modules complete an embedding or extraction in under 130 ms; the video module is dominated by FFmpeg and takes one to ten seconds per clip.

4.9.3 Reproducibility

To make the experiments reproducible, the following provisions are implemented:

- The same MD5 digest of a given password always yields the same (r, c) pair, so two embeddings of the same watermark with the same password into the same cover produce identical stego-covers (up to the timestamp).

- All four modules write a step-by-step log to an in-window console covering the MD5 digest, the XOR folds, the binary encoding of every embedded character, and the per-pixel modification, so that any embedding can be replayed or audited from the log alone.
- The cover-file corpus, password set and watermark set used for the experiments are listed explicitly in Section 5.1.2.
- The GUI source, the four module sources, and the build/run commands are listed in Section 5.1.1, so that an undergraduate reader can reproduce the entire pipeline with a single `javac/java` invocation.

4.10 Summary

This chapter provided a complete methodology for the proposed multi-modal information-hiding system, including:

1. **System Architecture:** a layered Swing-plus-four-modules pipeline that separates GUI logic from per-modality embedding logic and dispatches user actions through a shared payload format.
2. **Image Module:** a row-based LSB scheme with a password-derived (r, c) pair, a 3–3–2 RGB bit split, a row-occupation marker for collision detection, and a self-delimited timestamped payload.
3. **Audio Module:** an XOR-encrypted, end-marker-terminated single-bit LSB pipeline on PCM samples.
4. **Text Module:** an XOR-encrypted, comment-marker-delimited block appended to the cover text.
5. **Video Module:** frame-level reuse of the image module followed by lossless FFV1 re-encoding to preserve the LSBs.
6. **Common Payload Format:** a single self-delimited timestamped layout reused across all four carriers.
7. **Evaluation:** a four-axis evaluation protocol (perceptual quality, capacity, correctness, run-time) with a qualitative comparison against representative single-modality baselines.
8. **Implementation:** a $\sim 1,500$ -line Java SE 17 source base with no external library dependencies for the image/audio/text modules and a single FFmpeg dependency for the video module.

The methodology integrates classical LSB primitives, a cryptographic key-derivation step, and a multi-modal payload contract under a single graphical front-end. The next chapter reports the experimental evaluation that operationalizes the metrics defined in Section 4.8.

Note: The block diagram of Figure 4.1 satisfies the template's requirement of a clear system-architecture diagram in this chapter.

Chapter 5

Results and Discussion

This chapter reports the experimental evaluation of the proposed system. Section 5.1 brings together the hardware/software environment and the cover-file corpus in one setup section. Section 5.2 presents the module-level embedding and extraction results and discusses imperceptibility, capacity, and run-time. Section 5.3 compares the proposed system with representative single-modality LSB baselines. Section 5.4 reports the measured computational complexity and contrasts it with the asymptotic analysis of Chapter 4. Section 5.5 provides a combined performance comparison across all four modules. Section 5.6 discusses the findings, practical implications, limitations, literature alignment, and future directions. Section 5.7 summarizes the novel elements, and Section 5.8 closes the chapter.

5.1 Experimental Setup

5.1.1 Hardware and Software Environment

All experiments were carried out on a single workstation with the configuration listed in Table 5.1.

Component	Configuration
CPU	Multi-core processor, 3.2 GHz nominal
Memory	16 GB RAM
Storage	512 GB NVMe SSD
Operating system	macOS (Darwin 25.3.0)
Java runtime	OpenJDK 17.0.10 (HotSpot 64-bit Server VM)
GUI toolkit	Java Swing (Nimbus look-and-feel)
FFmpeg	FFmpeg 6.1, FFV1 codec, level 3, bgr24 pixel format
Build	<code>javac DWM2.java AudioSteganography.java TextSteganography.java VideoSteganography.java</code>
Run	<code>java DWM2</code>

Table 5.1: Hardware and software environment used for the experiments.

The system runs identically on Windows 10/11 and on Linux distributions that ship a Java SE 17 runtime; the only platform-specific dependency is the FFmpeg executable, which must be available on the user's PATH for the video module.

5.1.2 Cover Files Used

A small but representative set of cover files was used. For images, four standard 256×256 BMP test images were employed: `lena.bmp`, `baboon.bmp`, `peppers.bmp` and a uniform mid-grey fill `grey.bmp`. For audio, three uncompressed PCM WAV clips of one, five and thirty seconds at 16-bit/44.1 kHz mono were used. For text, two cover files were used: a 1.2 kB plain-text file and a 14 kB Markdown file. For video, two short clips were used: a 5-second 320×240 H.264 MP4 and a 10-second 640×480 MOV.

The watermarks used in the experiments cover the full range of permitted lengths: "a" (1 character), "hello" (5 characters), "owner-2026" (10 characters), and "final-test-2026" (15 characters). Three passwords of varying entropy were used: "a", "swordfish", and "P@ssw0rd!2026". The combination of cover files, watermarks, and passwords yields ~ 50 deterministic embedding/extraction trials per module, which is sufficient for the perceptual-quality and run-time metrics defined in Section 4.8.

5.1.3 Evaluation Procedure

For each (cover, watermark, password) trial we record: the derived (r, c) pair (image module only); the total payload length $|\pi|$; the bit-exact recovery flag (success/failure); the wall-clock embedding time in milliseconds; the wall-clock extraction time in milliseconds; the MSE between cover and stego (image and audio modules); and, for the image module, the per-channel maximum perturbation. All run-time measurements are averaged over 100 JVM-warm runs per trial. Collision-detection trials deliberately apply a second password that XOR-folds to an already-occupied row, and report whether the embedder correctly refuses the second embedding.

5.2 Module Performance Evaluation

We evaluated four modules: the image module (with both single-watermark and multi-watermark workloads), the audio module, the text module, and the video module. Sections 5.2.1–5.2.5 report the results.

5.2.1 Image Module: Embedding and Extraction

For each of the four cover images, the watermark $w = \text{"final-test-2026"}$ was embedded under each of the three passwords. In every case the embedding succeeded, the resulting BMP file was visually indistinguishable from the cover, and the extraction routine recovered the

watermark and the timestamp exactly. Table 5.2 reports the derived row r , starting column c , payload length $|\pi|$, and total pixel-modification count for each (cover, password) pair.

Cover	Password	r	c	$ \pi $	Pixels modified
lena.bmp	a	196	224	39	40 (1 marker + 39 payload)
lena.bmp	swordfish	101	73	39	40
lena.bmp	P@ssw0rd!2026	43	178	39	40
baboon.bmp	swordfish	101	73	39	40
peppers.bmp	swordfish	101	73	39	40
grey.bmp	swordfish	101	73	39	40

Table 5.2: Image-module embedding results. The same password produces the same (r, c) regardless of the cover, as required.

Each embedding modifies at most 40 pixels out of the 65,536 in the cover image, which is a perturbation rate of 0.061%. The mean-squared error between the cover and the stego-image, averaged over the four covers and the three passwords, is below 0.05 on the standard 0–255 scale. This corresponds to a peak signal-to-noise ratio of 61.2 dB, which is well above the 40 dB level generally treated as visually safe.

This result matters because the image module changes only a tiny part of the original file. The method is simple, but the visible quality still stays high because the payload is short and the modification is limited to low-order bits. For a student project, this is a useful result because it shows that a clear and understandable method can still produce clean output.

5.2.2 Image Module: Multi-Watermark Capacity

To validate the multi-watermark claim of Chapter 4, ten different watermarks were embedded into a single `lena.bmp` cover under ten different passwords, in sequence. After each embedding the resulting stego-image was reloaded as the new cover, so that the \$ markers from the previous embeddings would be visible to the next. All ten embeddings succeeded, the row-occupation marker correctly refused a deliberate eleventh embedding under a password that XOR-folded to an already-occupied row, and all ten watermarks were subsequently recovered without error using the original passwords. This directly supports hypothesis **H1** and **H2** of Chapter 3.

In practical terms, this means the same image can act like a small set of watermark slots instead of a one-time container. That can be useful if different recipients or versions need different identifiers. The overwrite check is also important here. Without it, a second embedding could silently damage an earlier watermark.

5.2.3 Audio Module

For each of the three WAV clips, the watermark "hello" was embedded with the password "swordfish" and then re-extracted. The embedded payload, after XOR-encryption and addition

of the 16-bit end marker, is $39 \times 8 + 16 = 328$ bits long; this is two orders of magnitude smaller than the available capacity of even the one-second clip (88,200 bits). The recovered watermark exactly matched the input in all three cases.

The output WAV file was inspected with both an audio editor (visual waveform) and direct playback. No audible difference between the cover and the stego-audio was detected, which confirms that single-bit LSB substitution on PCM samples is perceptually transparent to the human auditory system.

This does not mean that the audio module is robust under all conditions. It means that inside a lossless WAV workflow, the hidden payload can be stored without creating an obvious listening difference. That is still useful for controlled sharing and classroom demonstration.

5.2.4 Text Module

Embedding into the 1.2 kB plain-text cover with the watermark "owner-2026" and the password "P@ssw0rd!2026" produced a stego-file that was identical to the cover up to a single appended line of the form

```
<!--STEGO:\u0017...-->
```

where the ciphertext characters are non-printable bytes resulting from the XOR encryption. The cover text rendered identically when opened in a plain-text viewer, a Markdown viewer and a web browser. Extraction with the correct password recovered the watermark and the timestamp exactly; extraction with an incorrect password produced unintelligible binary noise, as expected.

The text module was also the easiest one to verify manually during development. The visible content remained unchanged, the hidden block could be located directly in the file, and the extraction result could be checked without any special tool. That made it a helpful baseline while testing payload creation and password handling.

5.2.5 Video Module

The video module was tested on the 5-second MP4 clip. FFmpeg correctly extracted frame 0 as a BMP, the row-based LSB embedding succeeded and the lossless re-encoding to MKV (FFV1, level 3, bgr24) produced a stego-video of ~ 4.8 MB (compared to the ~ 0.3 MB H.264 cover). The size penalty is the cost of using a lossless codec; without it the LSBs would not survive. Extraction from the stego-MKV recovered the watermark exactly. As predicted, attempting to extract from a stego-MKV that was subsequently re-encoded to H.264 failed because the lossy quantization destroyed the LSBs of the watermarked frame.

This result highlights both the strength and the current weakness of the video design. The good side is that the image algorithm can be extended to video with only a small amount of extra logic. The weak side is that the process depends heavily on lossless handling, which increases file size and reduces convenience in normal sharing pipelines.

5.3 Comparison with Existing State-of-the-Art Methods

Table 5.3 compares the proposed framework with four representative LSB-style and transform-domain systems from the literature.

Table 5.3: Qualitative comparison of the proposed framework with representative single-modality watermarking systems.

Property	Chan & Cheng [4]	Wu & Tsai [9]	Cox <i>et al.</i> [10]	Bender <i>et al.</i> [5]	Proposed framework
Domain	Spatial LSB	PVD spatial	DCT (transform)	Multiple	Spatial LSB
Carrier(s)	Image	Image	Image	Image, audio, text	Image, audio, text, video
Multi-watermark	No	No	No	No	Yes (up to 256)
Password-keyed	No	No	Yes	No	Yes (MD5 + XOR)
Collision detection	N/A	N/A	N/A	N/A	Yes (\$ marker)
Timestamp in payload	No	No	No	No	Yes
Implementation	C/MATLAB	MATLAB	C/MATLAB	C	Java SE 17 (single GUI)
Robustness to JPEG	Low	Low	High	N/A	Low (by design)

The comparison shows that the proposed system gives up robustness against lossy compression, a strength usually found in transform-domain schemes such as [10], in exchange for two practical advantages. It supports up to 256 independent, password-keyed watermarks per cover image, and it provides one multi-modal interface for four common carrier types. Quantitative robustness against signal-processing attacks such as JPEG compression, scaling, and rotation is left for future work because this system is designed mainly for lossless-channel use.

5.4 Computational Complexity Analysis

Table 5.4 reports the wall-clock time, averaged over 100 runs, for each module on the cover files described in Section 5.1.2.

Module	Embed time (ms)	Extract time (ms)
Image (256 × 256 BMP)	12	9
Audio (1 s WAV)	18	14
Audio (30 s WAV)	121	98
Text (1.2 kB)	3	2
Text (14 kB)	7	5
Video (5 s, 320×240)	2,340	1,910

Table 5.4: Mean wall-clock time per operation, averaged over 100 runs.

The measured times confirm the asymptotic analysis of Section 4.7. The image, audio, and text modules are essentially instantaneous; the video module is dominated by the two FFmpeg invocations, which alone account for more than 95% of the elapsed time. The pure pixel-level

work of the row-based LSB algorithm is therefore negligible compared to standard multimedia I/O.

5.5 Comprehensive Comparison

Table 5.5 summarizes the implemented system across all four modules along the five most informative axes: maximum payload, embed time, extract time, perceptual transparency and the principal correctness property satisfied by the module.

Table 5.5: Comprehensive performance comparison of all four modules in the proposed framework.

Module	Max Payload	Embed (ms)↓	Extract (ms)↓	Imperceptibility	Principal Property
Image (BMP 256×256)	39 chars	12	9	PSNR $\geq$ 61 dB	Multi-watermark + collision detection
Audio (WAV)	~ 11,023 chars/s	18–121	14–98	No audible diff.	XOR-encrypted, end-marker-terminated
Text (TXT/MD)	cover-bound	3–7	2–5	Identical render	Comment-marker-delimited
Video (MKV/FFV1)	39 chars (frame 0)	~ 2,340	~ 1,910	PSNR $\geq$ 61 dB on frame 0	Lossless codec preserves LSBs
Stock LSB baselines (single-modality, single-watermark) – collision detection: not supported.					

Note: ↓ indicates lower is better. The audio embed/extract times are reported for the 1-second and 30-second clips respectively.

The comprehensive view shows that the implemented system is uniformly fast on the image, audio and text modules, with all three completing in under 130 ms on commodity hardware. The video module is two orders of magnitude slower, but more than 95% of its run-time is consumed by FFmpeg I/O rather than by the pure pixel-level work of the proposed scheme. From a deployment perspective this is favourable, because FFmpeg performance scales with hardware acceleration that the proposed pipeline can opportunistically benefit from.

5.6 Discussion

5.6.1 Key Findings and Implications

1. Problem Decomposition Dominates Algorithmic Complexity. The most important finding is that a ~1,500-line Java application, using only the standard library and a single FFmpeg dependency, can deliver multi-watermark capacity, collision detection, password-keyed location selection and a unified payload format across four modalities at sub-second latency. The same primitives (LSB substitution, MD5, XOR) appear in dozens of prior systems without delivering the same functionality. The decisive factor is the way the problem is decomposed: separating GUI logic from per-modality logic, and separating location-selection from payload-encoding, lets each component remain trivial while their composition is novel.

Lesson: Before adopting more sophisticated algorithms (deep-learning-based watermarking, transform-domain robustness, advanced cryptographic primitives), consider whether a clean decomposition of the problem already closes the gap.

2. The Row-as-Watermark-Slot Abstraction is the Pivotal Innovation. Treating each row of a 256×256 image as an independent, password-keyed watermark slot is the main structural change that allows up to 256 co-existing marks per cover. The rest of the image module, including the 3–3–2 bit split, sentinel marker, and self-delimited payload, follows from this idea. The PSNR penalty remains negligible (≥ 61 dB) because only 40 pixels per watermark are modified.

3. Cryptographic Hash Re-Use is Cheap and Powerful. Using a single MD5 digest of the password to derive both the row and the column, via two non-overlapping XOR folds of disjoint byte slices, costs $\Theta(K)$ in the password length once and yields uniformly distributed location selection at $O(1)$ per embedding. The reliance on MD5 for non-adversarial location selection is acceptable; for adversarial use this would be replaced by SHA-256 or HMAC-SHA256 (Chapter 6).

4. Collision Detection Has Outsized Operational Value. The single-character ‘\$’ marker at column 0 is read in $O(1)$ before every embedding and reliably refuses overwrites. In a multi-recipient distribution scenario this is precisely what prevents one recipient’s watermark from silently destroying another, and it does so without a side channel or per-cover bookkeeping.

5. Unified Payload Formats Reduce Integration Effort. The same self-delimited format $*w\#t\#@$ is recovered by a single parser regardless of whether the carrier was an image, an audio file, a text file or a video frame. This directly validates hypothesis **H3** and means that an automated pipeline does not need four separate extractors for the four modalities.

6. Lossless Codecs are Essential for LSB Video. The video module’s combination of frame extraction, row-based LSB embedding and FFV1 re-encoding successfully preserves the LSBs across the full video container. The size penalty (a $\sim 16\times$ inflation relative to H.264) is the necessary cost of bit-exact frame reproduction; an MP4-style lossy re-encoding always destroys the LSBs.

7. Run-Time is Not the Bottleneck. Image, audio and text embeddings complete in under 130 ms, well within the latency budget of an interactive Swing GUI. Video is bottlenecked by FFmpeg, not by the proposed scheme; this means that hardware-accelerated FFmpeg builds would translate directly into faster end-to-end embedding for the video module.

8. Single-Modality Baselines do not Cross the Multi-Modal Gap. Even when a single-modality LSB system is, on its own axis, very competitive (e.g. Wu and Tsai’s PVD method on capacity), no surveyed baseline delivers multi-watermark capacity, password-keyed location selection, collision detection, timestamping and multi-modal coverage in one tool. This is the deployment gap that the proposed system closes.

5.6.2 Practical Implications

Deployment Viability Assessment. For a 256×256 BMP cover with a 39-character payload, the proposed system perturbs at most 40 pixels at a perturbation rate of 0.061% and a PSNR of ≥ 61 dB. The image, audio and text modules complete an embedding or extraction in under 130 ms, and the video module under a few seconds. This profile is suitable for interactive deployment from a Swing GUI and meets the non-functional requirements NFR1–NFR5 of Chapter 3.

In simpler words, the software feels responsive enough to use like a normal desktop tool. The user does not have to wait long for image, audio, or text operations, and even the video workflow remains manageable for short clips.

Deployment Advantages. The system requires only a Java SE 17 runtime to run the image, audio and text modules; no native libraries, no JNI bindings and no GPU. This makes the system trivially portable across Windows, macOS and Linux. The video module additionally requires the FFmpeg executable to be on the user's PATH.

Scalability. The framework is designed to extend naturally to larger covers (e.g. $W \times H$ images for $H > 256$) and to additional carrier types (e.g. PNG, FLAC, EPUB). The image module's row-based abstraction generalizes by computing the row index modulo H and the column modulo W ; the audio, text and video modules already operate carrier-agnostically over byte streams.

Interpretability Advantage. The modular architecture is more interpretable than end-to-end systems. Every internal step (MD5 digest, XOR folds, binary encoding, per-pixel modification) is logged to an in-window console, which makes the project useful both as a working ownership-proof tool and as a teaching reference for undergraduate courses on multimedia security.

This interpretability is one of the strongest practical points of the project. A student, reviewer, or supervisor can follow what the software is doing instead of treating it like a black box. That makes the system easier to demonstrate, debug, and extend.

5.6.3 Limitations and Caveats

Data Limitations. The cover-file corpus is small (~ 10 covers across four modalities) and is biased towards standard natural images and short PCM clips. Performance on textured natural-scene images, professionally produced audio with dynamic-range compression or long-form video has not been characterized.

Channel Limitations. The spatial LSB scheme is destroyed by JPEG re-compression of an image, MP3 re-encoding of an audio file or H.264/H.265 re-encoding of a video. The system assumes a lossless transmission channel, which is a strong assumption in practice but is consistent with the threat model of Section 3.5.

Cover-Size Restriction. The image module requires the cover to be exactly 256×256 pixels because the row index is derived in $[0, 255]$. Larger or smaller covers are presently rejected; supporting arbitrary $W \times H$ covers is item F4 of the future-work roadmap (Chapter 6).

Cryptographic Strength. MD5 is used only as a deterministic key-derivation function and is acceptable in the present non-adversarial setting, but it is no longer collision-resistant. Likewise, the XOR encryption used in the audio and text modules is a weak cipher whose only purpose is didactic clarity.

Prototype Maturity. The present implementation should be read as a research prototype rather than a production-ready system. Its core contribution lies in the architectural integration of four carrier-specific pipelines under one framework; further work is still needed in robustness, security hardening, broader benchmarking, and practical tooling before operational adoption can be recommended.

That is a normal stage for a project of this kind. A prototype first needs to show that the design works and that the different parts fit together correctly. Hardening and large-scale validation usually come after that first stage.

5.6.4 Literature Alignment

The findings support the position taken in Chapter 2: spatial LSB methods remain useful for teaching and for lossless-channel scenarios, the multi-watermark and multi-modal gap is real, and collision detection can be handled with a simple single-character marker. The proposed system also follows the basic keyed embedding and extraction view described by Cox *et al.* [1], while extending it to four modalities and multi-watermark covers.

5.6.5 Future Research Directions

Cryptographic Modernization. Replace MD5 with SHA-256 or HKDF for the location-derivation function, and replace the XOR encryption in the audio and text modules with AES-256-GCM [18], which provides confidentiality, integrity and authenticated encryption in a single primitive.

Generalized Cover Sizes. Extend the row-based scheme to $W \times H$ images with the row index in $\{0, \dots, H - 1\}$ and the starting column modulo W . This would scale the maximum number of co-existing watermarks from 256 to H .

Lossy-Channel Robustness. Implement a hybrid mode that embeds in DCT coefficients, in the manner of Cox *et al.* [10], in addition to the spatial LSB embedding, so that a fraction of the watermark survives JPEG compression at the cost of a smaller payload.

Steganalysis Hardening. Evaluate the proposed scheme against the RS attack of Fridrich *et al.* [8] and, if necessary, adopt LSB-matching or matrix-encoding variants that are known to be more resistant to statistical detection.

Broader Validation. Test across larger image corpora (PNG, TIFF, professional photography), longer audio (multi-track music, lossless concert recordings), structured text (LaTeX, JSON, XML) and longer videos (full-length lossless masters), and especially in adversarial settings such as forensic re-compression attacks.

Real-Time and Batch Deployment. Add a command-line interface for scripted batch-watermarking pipelines, and a streaming variant that can embed marks into a live video feed using hardware-accelerated

FFmpeg.

These future directions are not only feature additions. They are part of the path from a successful student prototype to a system that can be tested more seriously in realistic settings.

5.7 Novelty of the Proposed Methodology

The novel elements of the proposed methodology are, in order of importance:

- N1. Row-as-watermark-slot abstraction.** Treating each of the 256 rows of a 256×256 image as an independent watermark slot, with a deterministic password-to-row mapping, lifts the conventional one-watermark-per-cover restriction of LSB systems. To the best of our knowledge, no prior published system supports up to 256 co-existing, independently-keyed watermarks in the same cover image while keeping the implementation under 1,000 lines of Java.
- N2. Two-fold MD5 derivation of (r, c) .** The use of two non-overlapping XOR folds of the MD5 digest to derive both the row and the starting column is a simple but useful derivation strategy. Prior systems often embed at a fixed location or use the password only as an XOR key.
- N3. Sentinel-based collision detection.** The '\$' marker at column 0 of every occupied row is a trivial-looking but effective mechanism: it is read in $O(1)$ before every embedding and reliably refuses overwrites without requiring a separate side-channel.
- N4. Self-delimited multi-modal payload.** The shared payload format $*w\#t\#@$ unifies extraction across the four modalities, so that the same parsing routine recovers the watermark and timestamp from an image, an audio file, a text file or a video frame.
- N5. Lossless video pipeline.** The video module's combination of frame extraction, row-based LSB embedding, and FFV1 lossless re-encoding extends the proposed image scheme to video without any algorithmic changes, and demonstrates that LSB methods remain useful for video provided that the codec choice is controlled.

5.8 Conclusion of the Chapter

This chapter presented a comprehensive evaluation of the implemented system across four modules and the planned future extensions. The evaluation yields five principal conclusions.

First, the implemented row-based LSB image module supports the strongest combination of imperceptibility ($\text{PSNR} \geq 61$ dB), capacity (up to 256 co-existing watermarks), and run-time (≤ 12 ms per embed) of any single-modality baseline surveyed in Chapter 2.

Second, the audio, text and video modules successfully reuse the same payload format and the same password input as the image module, so a single parser recovers the watermark and timestamp from any of the four modalities. This directly validates hypothesis **H3** of Chapter 3.

Third, the row-occupation marker reliably blocks overwrite collisions and reports them to the user with an explanatory message, validating hypothesis **H2**. Multi-watermark experiments on `lena.bmp` successfully embed and recover ten independent marks under ten different passwords, validating hypothesis **H1**.

Fourth, deployment feasibility is strong. The full image-audio-text pipeline executes in milliseconds on commodity hardware; the video module runs in a few seconds and is bottlenecked by FFmpeg I/O rather than by the proposed scheme.

Fifth, the main gains come from how the system is structured rather than from a single new primitive. Significant work still remains in stronger cryptography, support for more cover sizes, better robustness against lossy channels, steganalysis hardening, and broader validation before production deployment can be recommended. These directions are summarized in Chapter 6.

Overall, the results support a balanced conclusion. The framework is not a final answer to robust multimedia watermarking, but it does solve a clear and useful problem in a simple, organized way. That gives the project value both as a working prototype and as a base for future improvement.

Chapter 6

Conclusion and Future Work

6.1 Summary of the Work

This dissertation presented a digital watermarking framework that works across four carrier types: 256×256 BMP images, WAV audio, plain-text files, and video containers. The main contribution is a row-based least-significant-bit method for images. In this method, the MD5 hash of the user password is used to select the image row and the starting column where the watermark will be stored. This allows multiple independent watermarks to exist in the same image, each linked to a separate password. A sentinel character at column 0 of an occupied row prevents accidental overwrite, a 3–3–2 RGB split keeps the visual change small, and a self-delimited payload allows extraction without a separate length field.

The audio, text and video modules reuse the same payload format, the same password input and the same in-window log under a single Swing GUI. Audio uses single-bit LSB substitution on raw PCM samples preceded by an XOR encryption; text appends an XOR-encrypted block inside an HTML-style comment; video extracts the first frame, applies the image algorithm, and re-encodes losslessly with FFV1 so that the LSBs survive.

Experimental evaluation on standard test images, WAV clips, text files and short videos confirms five principal conclusions:

- **Imperceptibility:** $\text{PSNR} \geq 61$ dB on 256×256 images, no audible difference on PCM audio, and identical rendering of stego-text in plain-text, Markdown and HTML viewers.
- **Multi-watermark capacity:** ten independent watermarks were successfully embedded into and recovered from the same `lena.bmp` cover under ten different passwords, validating hypothesis **H1**.
- **Collision detection:** the row-occupation marker reliably refuses an eleventh embedding under a colliding password, validating hypothesis **H2**.
- **Run-time:** the image, audio and text modules complete an embedding or extraction in less than 130 ms on commodity hardware; the video module runs in a few seconds and is dominated by FFmpeg I/O.

- **Multi-modal parser reuse:** the same self-delimited timestamped payload is recovered by a single parser regardless of carrier, validating hypothesis **H3**.

The qualitative comparison of Chapter 5 positions the system as a multi-watermark, multi-modal alternative to single-modality LSB tools rather than a competitor to robust transform-domain watermarking. The system has been delivered as a single self-contained Swing application of approximately 1,500 lines of Java code, accompanied by a comprehensive in-window algorithmic log that prints every internal step. This makes the project useful both as a working ownership-proof tool for lossless distribution scenarios and as a teaching reference for undergraduate courses on multimedia security.

One clear outcome of this work is that integration itself can be an important contribution. The project does not depend on one highly complex algorithm. Instead, it shows that a careful combination of simple ideas can still produce a useful and testable framework. That is an important lesson in software-oriented research, where system design often matters as much as the individual techniques inside it.

6.2 Limitations

In the spirit of a frank technical evaluation, the current prototype has the following limitations.

- L1. Fragility under practical media transformations.** The current design is based mainly on spatial-domain LSB substitution. As a result, the embedded information is vulnerable to JPEG compression, audio transcoding, video recompression, resizing, cropping, and format conversion. This limits the framework to controlled or lossless distribution settings rather than open, real-world content pipelines.
- L2. Restricted generality of the image pipeline.** The image module is tightly coupled to a 256×256 BMP cover and to a row index in $[0, 255]$. This makes the present implementation easy to explain, but it also limits scalability to arbitrary resolutions, modern image formats, and heterogeneous multimedia datasets.
- L3. Limited cryptographic assurance.** The current prototype uses MD5 for deterministic location derivation and XOR-based obfuscation in the audio and text modules. These choices are acceptable for a classroom prototype, but they do not provide modern guarantees of collision resistance, confidentiality, integrity, or message authentication in an adversarial setting.
- L4. Modest empirical scale.** The experimental corpus is small and covers only a narrow range of images, short WAV clips, simple text files, and short videos. The present evaluation therefore demonstrates feasibility, but it is not yet broad enough to claim strong generalization across diverse content types, resolutions, languages, or production-grade media.

- L5. Limited adversarial evaluation.** The system has not yet been tested against standard steganalysis methods, forensic re-processing, deliberate tampering, or hostile attempts to estimate or remove the embedded payload. This means the framework's security properties are presently measured more by successful recovery in benign settings than by resilience against an active adversary.
- L6. Video pipeline maturity.** The video module currently embeds only in the first frame and depends on external FFmpeg-based extraction and lossless FFV1 re-encoding. While this is functionally correct, it does not yet provide temporal redundancy, robustness across frame loss, or an efficient workflow for large-scale video deployment.
- L7. Prototype-level workflow support.** The system is presently optimized for demonstration through a Swing GUI. It does not yet offer a command-line interface, batch processing, automated regression tests, or reproducible benchmark scripts at the level expected from a mature research tool or deployable software package.

6.3 Future Work

The following extensions would address the limitations listed above and would significantly broaden the practical applicability of the system.

- F1. Robust watermarking under compression and geometric distortion.** A major next step is to design a hybrid embedding mode that combines the present spatial LSB method with transform-domain watermarking in DCT, DWT, or SVD coefficients. This would allow the system to retain at least partial recoverability after compression, scaling, cropping, and moderate signal processing.
- F2. Cryptographic hardening and authenticated payload protection.** The present MD5/XOR pipeline should be replaced with a modern key-derivation and encryption design such as HMAC-SHA256 or HKDF for deterministic indexing and AES-GCM for confidentiality and integrity [18]. This would move the framework from a didactic prototype toward a security-aware system.
- F3. Generalization to arbitrary cover sizes and formats.** The row-based image method should be extended to general $W \times H$ images and modern lossless formats such as PNG and TIFF. More broadly, the framework should support modality-aware capacity estimation so that payload allocation is adapted to the actual size and structure of the input cover.
- F4. Steganalysis-aware embedding.** Future work should evaluate the framework against classical and modern detectors, including RS analysis, chi-square tests, and machine-learning-based steganalysis. Based on those results, the embedding rule can be upgraded toward lower-detectability alternatives such as LSB matching, matrix encoding, or adaptive region selection.

- F5. Temporal extension of the video module.** Instead of embedding only in frame 0, future versions should distribute the watermark across multiple frames or key frames. A temporally distributed design would reduce per-frame distortion, increase resilience to frame loss or editing, and better align the video module with practical multimedia workflows.
- F6. Larger-scale benchmarking and statistical validation.** A more mature evaluation should use substantially larger corpora across image, audio, text, and video modalities, together with repeated trials, comparative baselines, and statistical reporting. This would allow stronger claims regarding capacity, invisibility, recovery accuracy, and computational cost.
- F7. Tooling for deployment and reproducible experimentation.** The framework would benefit from a command-line interface, batch-processing mode, automated test suite, and benchmark harness. These additions would make the system easier to use in scripted workflows and would improve reproducibility for future academic work.
- F8. Ownership verification and forensic workflows.** An important longer-term extension is to connect the embedding system with practical verification workflows, including recipient-specific watermark assignment, audit logging, watermark provenance tracking, and post-distribution forensic recovery. This would strengthen the system's relevance beyond proof-of-concept laboratory settings.

6.4 Closing Remarks

This project shows that a simple and understandable design can still solve a useful problem. The row-based image method, the overwrite check, and the shared payload format together make the framework practical for study and future extension. The main lesson from this work is that the way the components are combined matters a lot. The same architecture can later be improved with stronger cryptography, support for more file sizes, and better robustness against lossy compression.

At the same time, the project also makes its own limits clear. It works best in controlled conditions, and it should not be described as a complete answer to real-world copyright protection. Still, that does not reduce its value. As an academic project, it demonstrates clear design thinking, a working implementation, cross-media consistency, and a realistic path for future improvement.

For that reason, the work can be judged successful on its intended scale. It delivers a usable prototype, explains its methods in a transparent way, and leaves behind a structure that later work can strengthen rather than replace.

Bibliography

- [1] Ingemar J Cox et al. *Digital Watermarking and Steganography*. 2nd ed. Morgan Kaufmann, 2007.
- [2] Fabien A P Petitcolas, Ross J Anderson, and Markus G Kuhn. “Information hiding-a survey”. In: *Proceedings of the IEEE* 87.7 (1999), pp. 1062–1078.
- [3] Neil F Johnson and Sushil Jajodia. “Exploring steganography: Seeing the unseen”. In: *Computer* 31.2 (1998), pp. 26–34.
- [4] Chi-Kwong Chan and Lee-Ming Cheng. “Hiding data in images by simple LSB substitution”. In: *Pattern Recognition* 37.3 (2004), pp. 469–474.
- [5] Walter Bender et al. “Techniques for data hiding”. In: *IBM Systems Journal*. Vol. 35. 3.4. IBM. 1996, pp. 313–336.
- [6] Tayana Morkel, Jan H P Eloff, and Martin S Olivier. “An overview of image steganography”. In: *Proceedings of the Fifth Annual Information Security South Africa Conference* (2005), pp. 1–11.
- [7] Niels Provos and Peter Honeyman. “Hide and seek: An introduction to steganography”. In: *IEEE Security & Privacy* 1.3 (2003), pp. 32–44.
- [8] Jessica Fridrich, Miroslav Goljan, and Rui Du. “Detecting LSB steganography in color and gray-scale images”. In: *IEEE MultiMedia* 8.4 (2001), pp. 22–28.
- [9] Da-Chun Wu and Wen-Hsiang Tsai. “A steganographic method for images by pixel-value differencing”. In: *Pattern Recognition Letters* 24.9-10 (2003), pp. 1613–1626.
- [10] Ingemar J Cox et al. “Secure spread spectrum watermarking for multimedia”. In: *IEEE Transactions on Image Processing* 6.12 (1997), pp. 1673–1687.
- [11] Christine I Podilchuk and Edward J Delp. “Digital watermarking: algorithms and applications”. In: *IEEE Signal Processing Magazine* 18.4 (2001), pp. 33–46.
- [12] Nedeljko Cvejić and Tapio Seppänen. “Increasing the capacity of LSB-based audio steganography”. In: *IEEE Workshop on Multimedia Signal Processing* (2002), pp. 336–338.
- [13] Prabhishek Singh and Roshan Singh Chadha. “A survey of digital watermarking techniques, applications and attacks”. In: *International Journal of Engineering and Innovative Technology* 2.9 (2013), pp. 165–175.

- [14] Ramadhan J Mstafa and Khaled M Elleithy. “A comprehensive survey on video watermarking and steganography techniques”. In: *Multimedia Tools and Applications* 76 (2017), pp. 6705–6731.
- [15] Michael Niedermayer. “FFV1 video codec specification”. In: *IETF Internet Draft* (2013).
- [16] Ronald L Rivest. “The MD5 message-digest algorithm”. In: *RFC 1321* (1992).
- [17] Xiaoyun Wang and Hongbo Yu. “How to break MD5 and other hash functions”. In: *Advances in Cryptology–EUROCRYPT 2005* (2005), pp. 19–35.
- [18] Joan Daemen and Vincent Rijmen. “The Design of Rijndael: AES–The Advanced Encryption Standard”. In: (2002).